Using Xrm.WebApi methods and rollup fields

In this lecture we will learn how to use JavaScript with Xrm for web API. Navigate to https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/reference/xrm-webapi to see a full reference about the topic we are going to discuss.

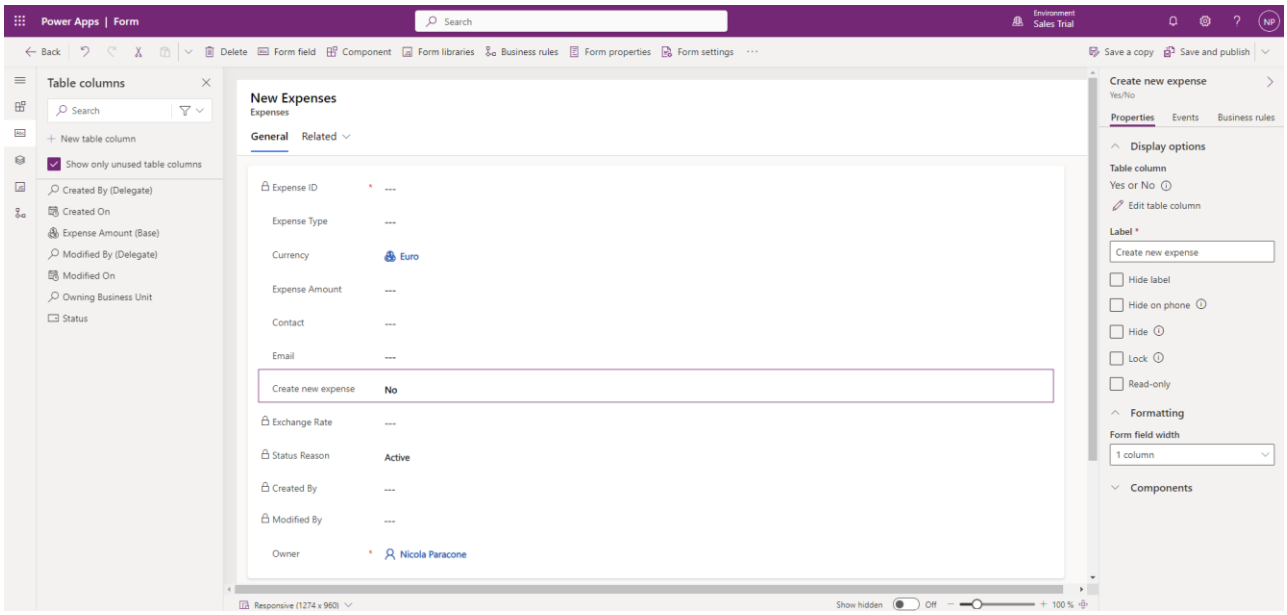
We already learnt that there are two types of custom codes: client-side scripting and server-side scripting. For client side scripting we will be using HTML, JavaScript, CSS (to design the web page with customized styles).

1. Creating, updating and deleting records using Xrm

The first thing we explore is how to create a record. Checking the documentation, the syntax for this purpose is Xrm.WebApi.createRecord(entityLogicalName,data).then(successCallback,errorCallback). Besides entityLogicalName, there is also entitySetName, which is the plural name of the entity. The following is the example provided in the Microsoft documentation.

```
// define the data to create new account
var data =
    {
        "name": "Sample Account",
        "creditonhold": false,
        "address1_latitude": 47.639583,
        "description": "This is the description of the sample account",
        "revenue": 5000000,
        "accountcategorycode": 1
    }

// create account record
Xrm.WebApi.createRecord("account", data).then(
    function success(result) {
        console.log("Account created with ID: " + result.id);
        // perform operations on record creation
    },
    function (error) {
        console.log(error.message);
        // handle error conditions
    }
);
```

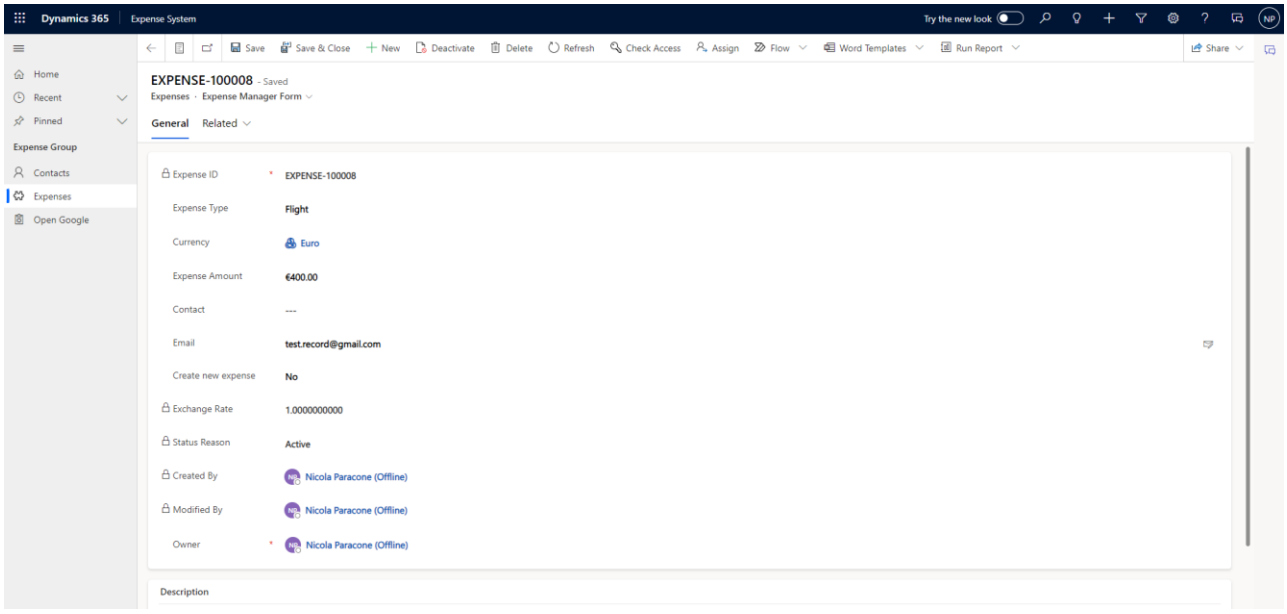
Notice that we have a method for success and one for error. In the following we modify the name of the Yes or No field inside the Expense Manager Form to Create new expense.



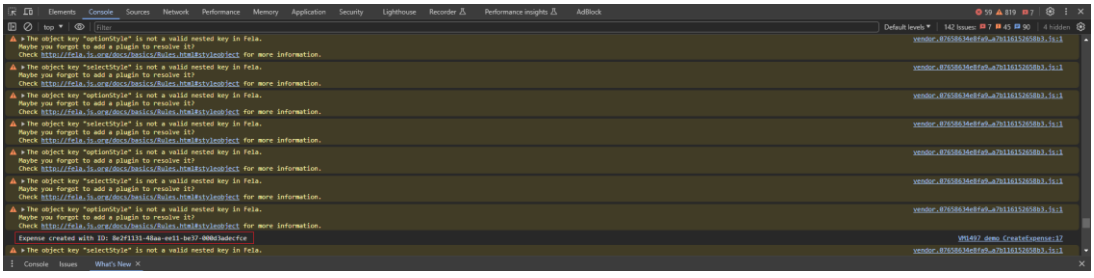
Hence, we write a code with this purpose: on change of the field Create new expense, one new expense (which is a record) is created. The code below, for the moment, creates a new expense on changing of the Create new expense field without taking into account any condition on its value.

```
function createExpense(executionContext) {

    debugger;

    // Define the data to create new expense
    var data =
    {
        "demo_expensetype": 100000000, // Which is Flight
        "demo_expenseamount": 400,
        "demo_email": "test.record@gmail.com"

    }

    // Create expense record
    Xrm.WebApi.createRecord("demo_expenses", data).then(
        function success(result) {
            console.log("Expense created with ID: " + result.id);
            // perform operations on record creation
        },
        function (error) {
            console.log(error.message);
            // handle error conditions
        }
    );
}
```

Contrary to our previous examples, no formcontext is required. Once the web resource is created and associated as event handler with Create new expense, the result we obtain in Dynamics 365 is a new expense having field values equal to those we have specified inside the code.



We can also add a debugger statement just below the function declaration. From the browser console we read if our expense creation has been successful or not. If it is successful, we can also read the associated Guid.

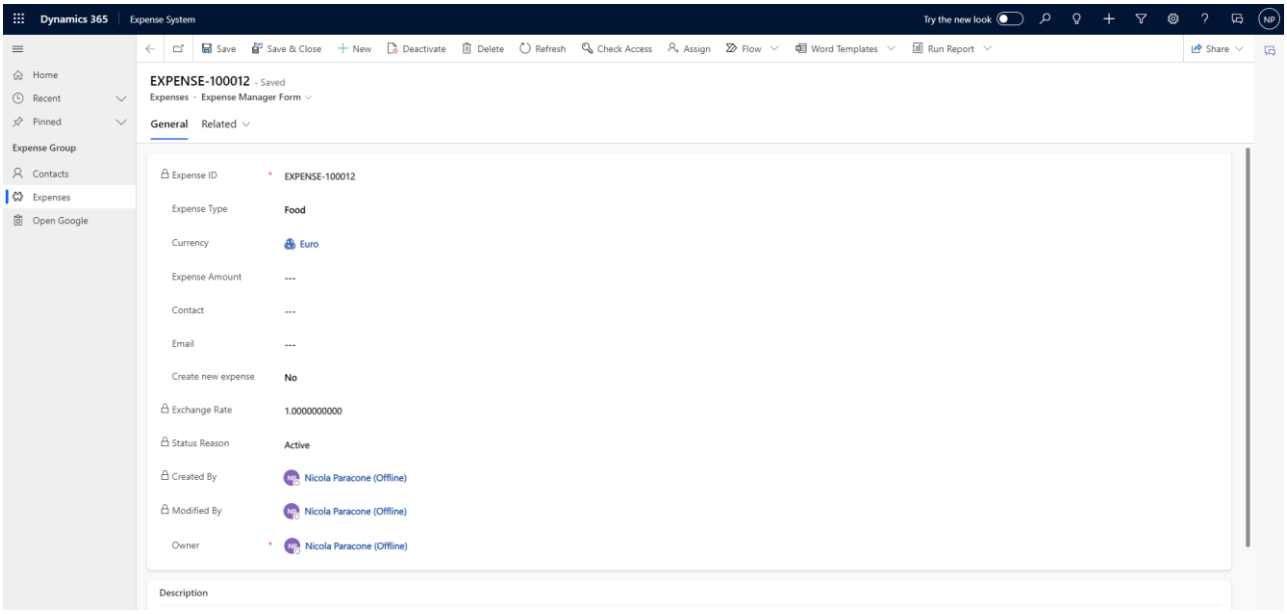


If we don’t have the time to check the console, we can retrieve the Guid by inserting a simple alert statement.

To update a record, the syntax we have to use is Xrm.WebApi.updateRecord(entityLogicalName, id, data).then(successCallBack, errorCallback). In this case we must pass also the Id of the record we are updating. The following is the example contained in the official documentation.

```
// define the data to update a record
var data =
    {
        "name": "Updated Sample Account ",
        "creditonhold": true,
        "address1_latitude": 47.639583,
        "description": "This is the updated description of the sample account",
        "revenue": 6000000,
        "accountcategorycode": 2
    }
// update the record
Xrm.WebApi.updateRecord("account", "5531d753-95af-e711-a94e-000d3a11e605", data).then(
    function success(result) {
        console.log("Account updated");
        // perform operations on record update
    },
    function (error) {
        console.log(error.message);
        // handle error conditions
    }
);
```

Suppose that we are trying to update the following expense.

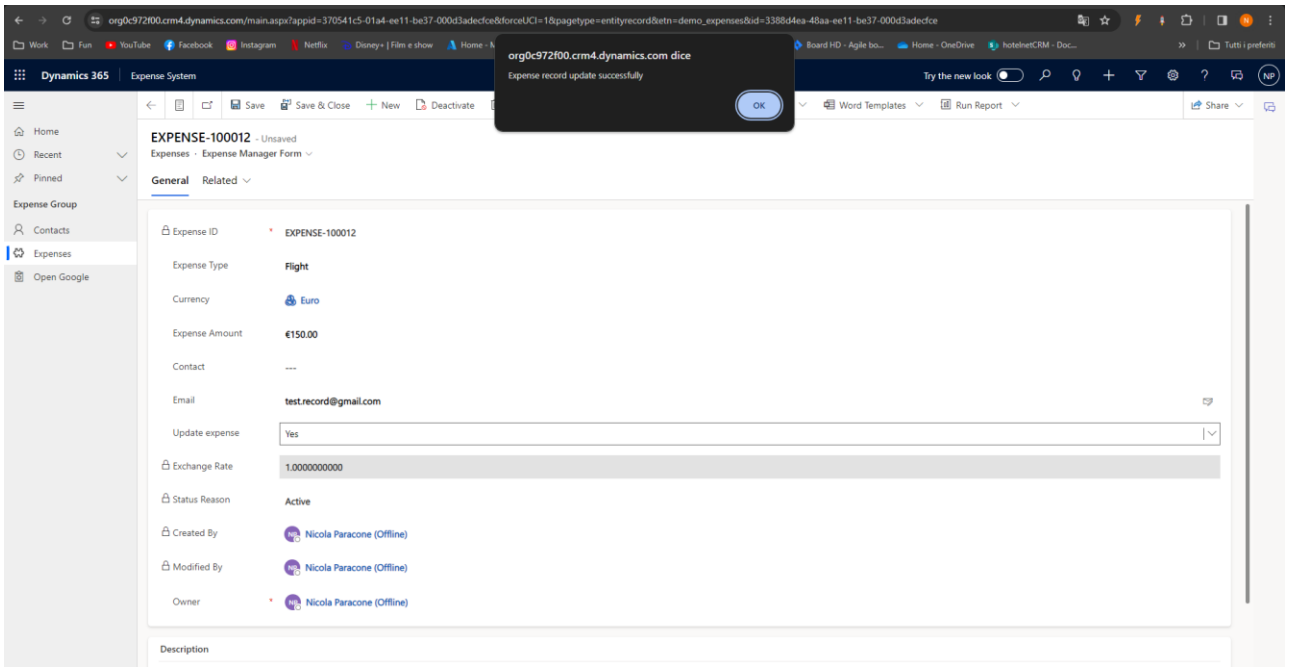


This has Id 3388d4ea-48aa-ee11-be37-000d3adecfce.

By means of the following code:

```
function updateExpense(executionContext) {

    debugger;

    var formcontext = executionContext.getFormContext();
    var expenseGuid = formcontext.data.entity.getId();
    // define the data to update an expense
    var data =
    {
        "demo_expensetype": 100000000,
        "demo_expenseamount": 150,
        "demo_email": "test.record@gmail.com"

    }
    // demo_expenses
    // update expense record
    Xrm.WebApi.updateRecord("demo_expenses", expenseGuid, data).then(
        function success(result) {
            alert("Expense record update successfully")
            console.log("Expense updated");
            // perform operations on record update
        },
        function (error) {
            alert(error.message);
            console.log(error.message);
            // handle error conditions
        }
    );
}
```

the resource acquires the Guid of the record currently visible in Dynamics 365, and on change of the Create new expense (feel free to rename it as Update expense) field it updates its content consistently with the values inside the data object. An alert message is displayed both for success and failure.

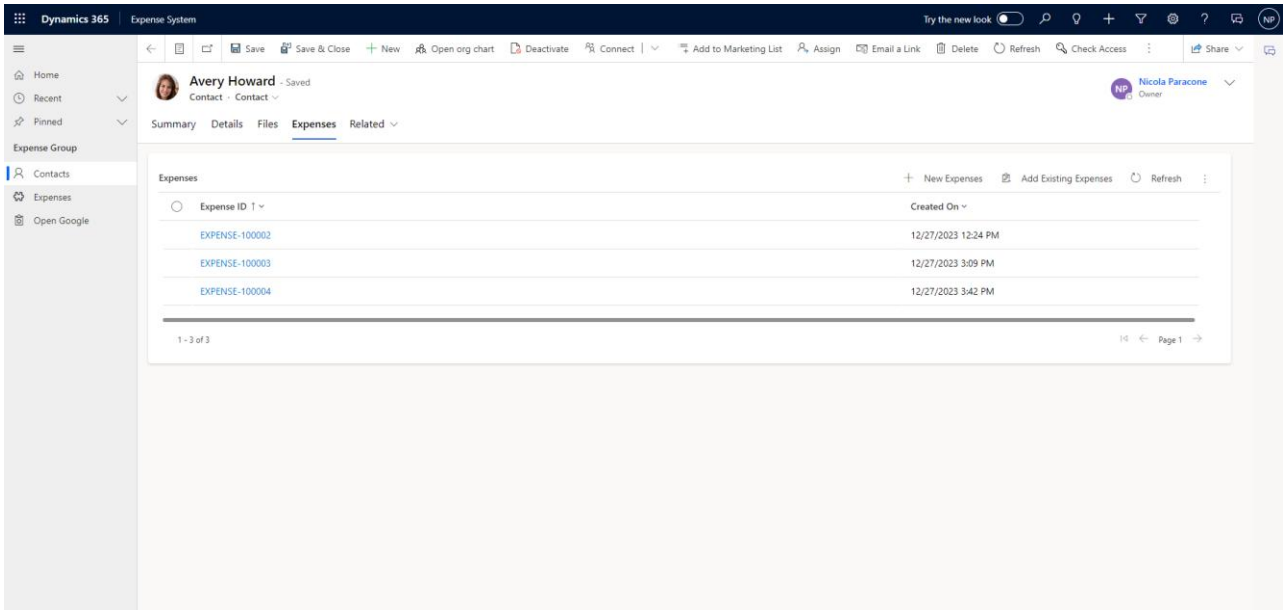


If we need to delete a record, the syntax is Xrm.WebApi.deleteRecord(entityLogicalName, id).then(successCallback, errorCallback). Notice that no data are passed, since for deleting a record we only need its Guid. The following is the example of the official documentation.

```
Xrm.WebApi.deleteRecord("account", "5531d753-95af-e711-a94e-000d3a11e605").then(
    function success(result) {
        console.log("Account deleted");
        // perform operations on record deletion
    },
    function (error) {
        console.log(error.message);
        // handle error conditions
    }
);
```

2. Rollup fields

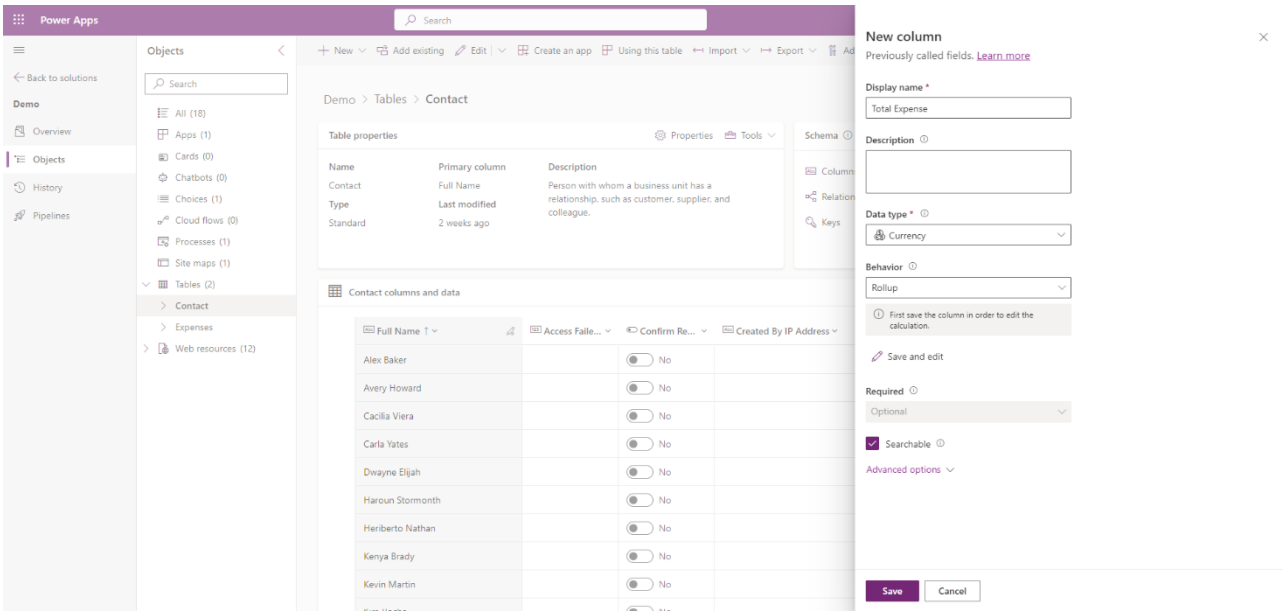
In some scenarios we may need to know how much expenses has paid a particular customer till now.



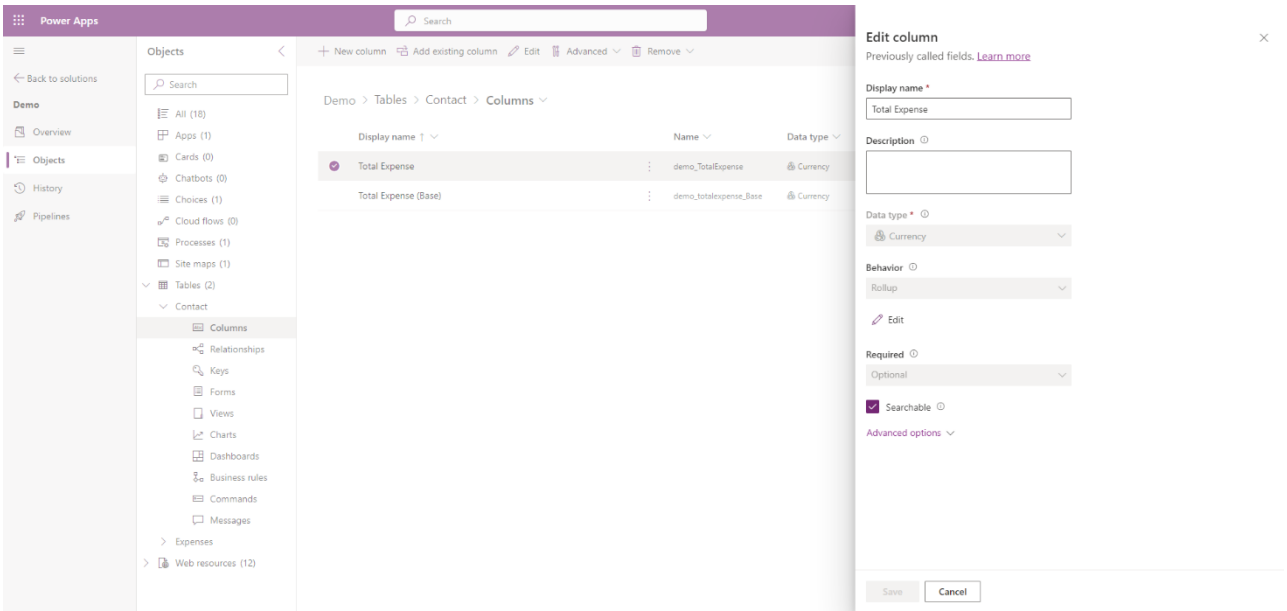
If we have many records, it is useful to create a rollup field calculating the sum of the amounts of all expenses in the contact entity.

A **rollup field**, in the context of databases and data management, is a calculated field that aggregates data from related records and displays a summary value on a parent record. It is commonly used in relational database systems to perform calculations on child records and summarize the results at the parent level.

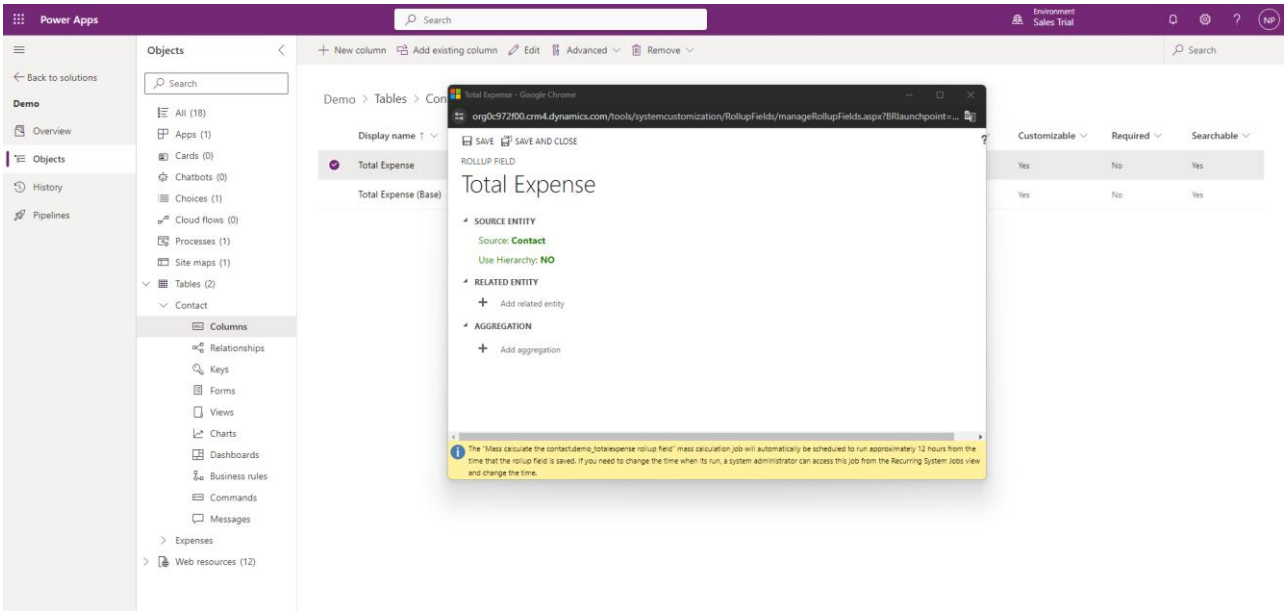
To do that we start by creating a new column in the Contact table.



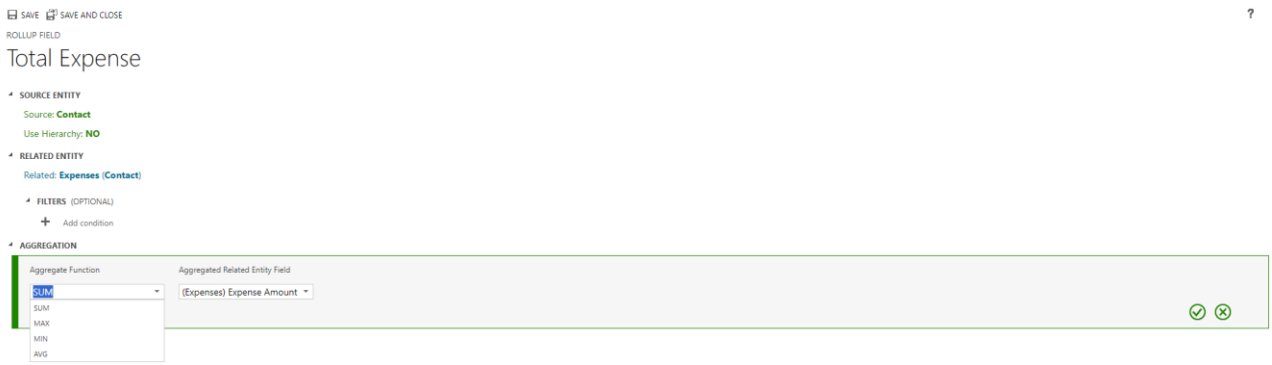
Remember that once the new column is created it will not be possible to edit its Data type, Behavior and Required fields.



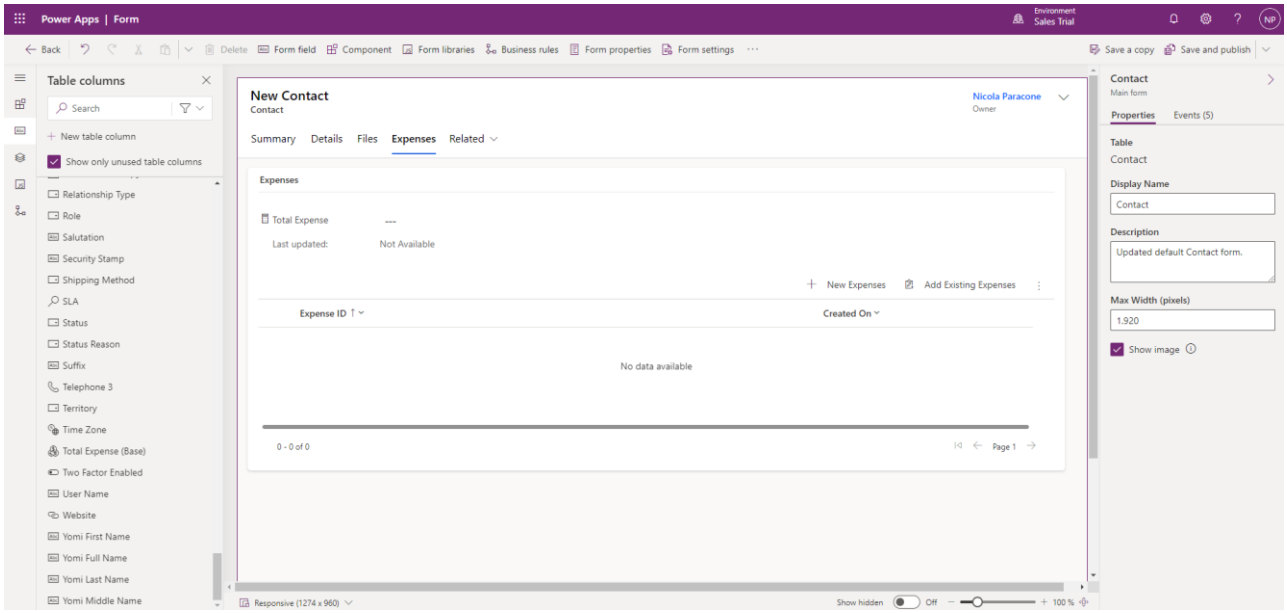
Now click on Edit below the Behavior choice. The following window will pop up.



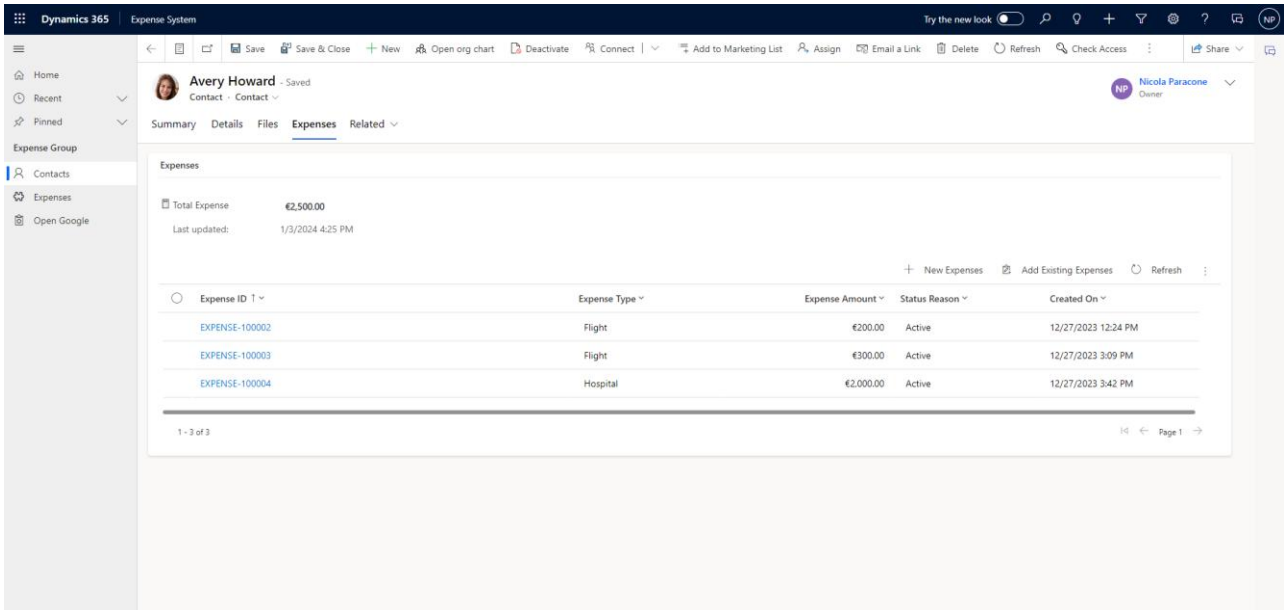
Here we have to define the related entity to which we apply the rollup field and the method of aggregation of its data.



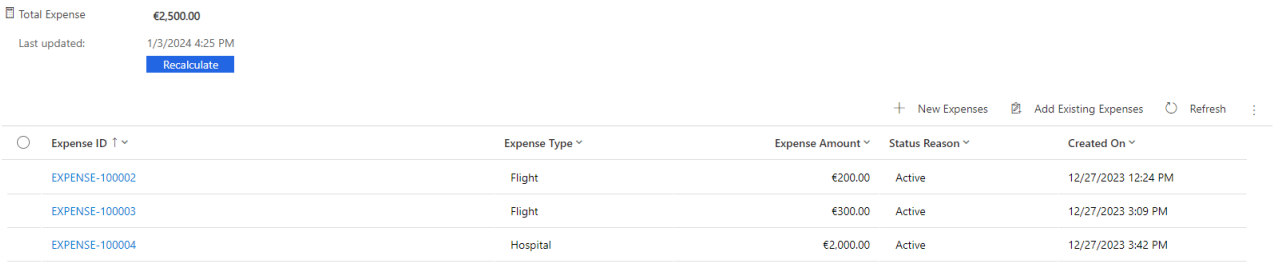
The next step consists in adding the newly created field to the Contact form.



The effect in the Expenses tab of a contact form is the following.



By clicking on the calculator symbol next to Total Expense, a Recalculate button will appear.



3. Get related records

Suppose that on change of the Email field in an expense we want to retrieve the related contact email and check if the two email addresses are equal or not. If they don't match, we want an alert to be thrown.

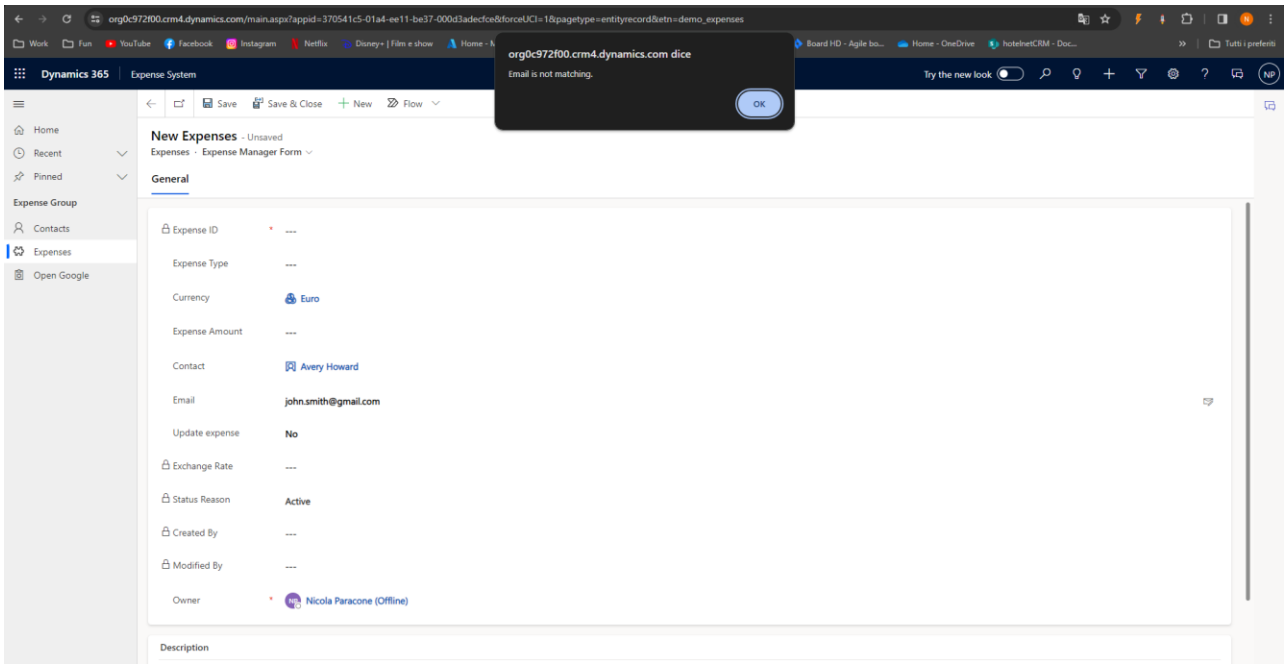
Navigate to https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/reference/xrm-webapi/retrieverecord for the full documentation.

The basic syntax for retrieving a record is Xrm.WebApi.retrieveRecord(entityLogicalName, id, options).then(successCallback, errorCallback).

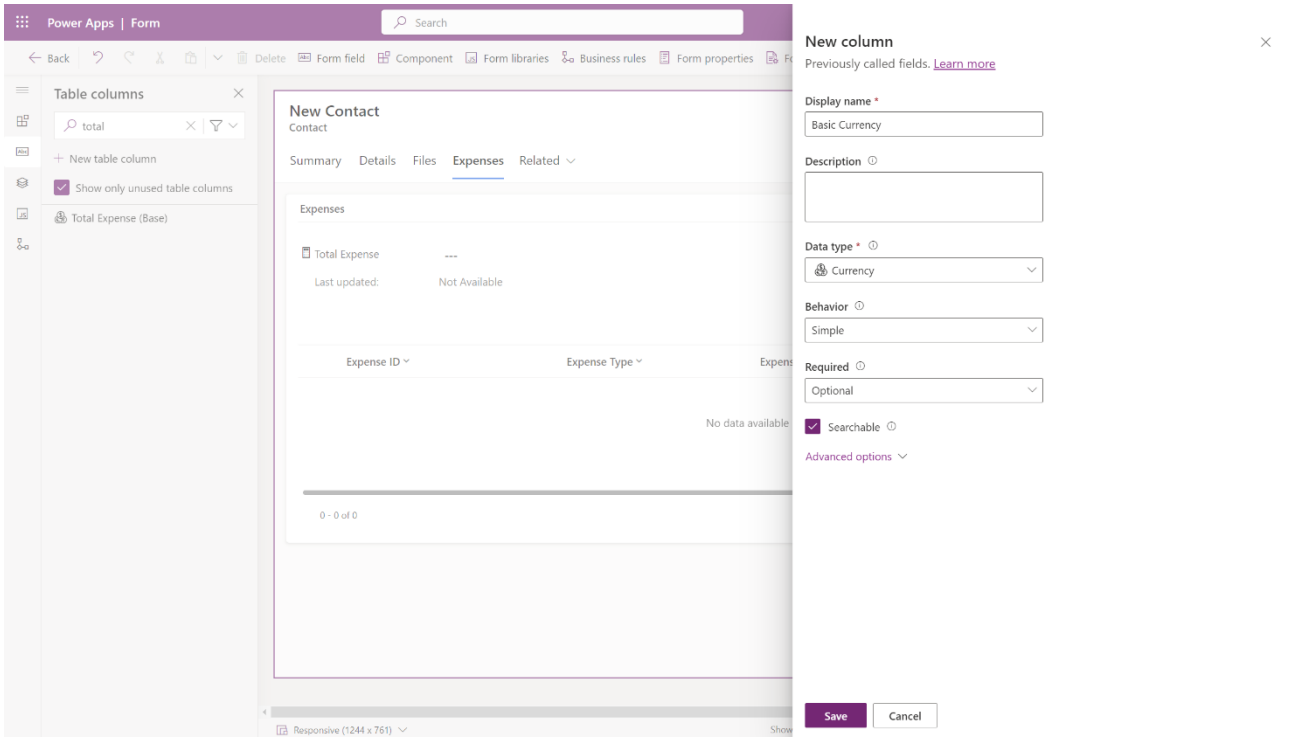
Let us a create the JavaScript web resource containing the following code.

```
function getRelatedRecord(executionContext) {

    debugger;
    var formcontext = executionContext.getFormContext();

    var contactID = formcontext.getAttribute("demo_contact").getValue()[0].id; // This is for obtaining the contact id in the Expense entity
    var email = formcontext.getAttribute("demo_email").getValue();   // This is for obtaining the email address in the Expense entity

    Xrm.WebApi.retrieveRecord("contact", contactID, "?$select=emailaddress1").then( // We retrieve the corresponding contact record in the Contact entity
        function success(result) {

            if (email != result.emailaddress1) {

                alert("Email is not matching.");

            }
            console.log("Retrieved values: Email: " + result.emailaddress1);
            // perform operations on record retrieval
        },
        function (error) {
            console.log(error.message);
            // handle error conditions
        }
    );
}
```

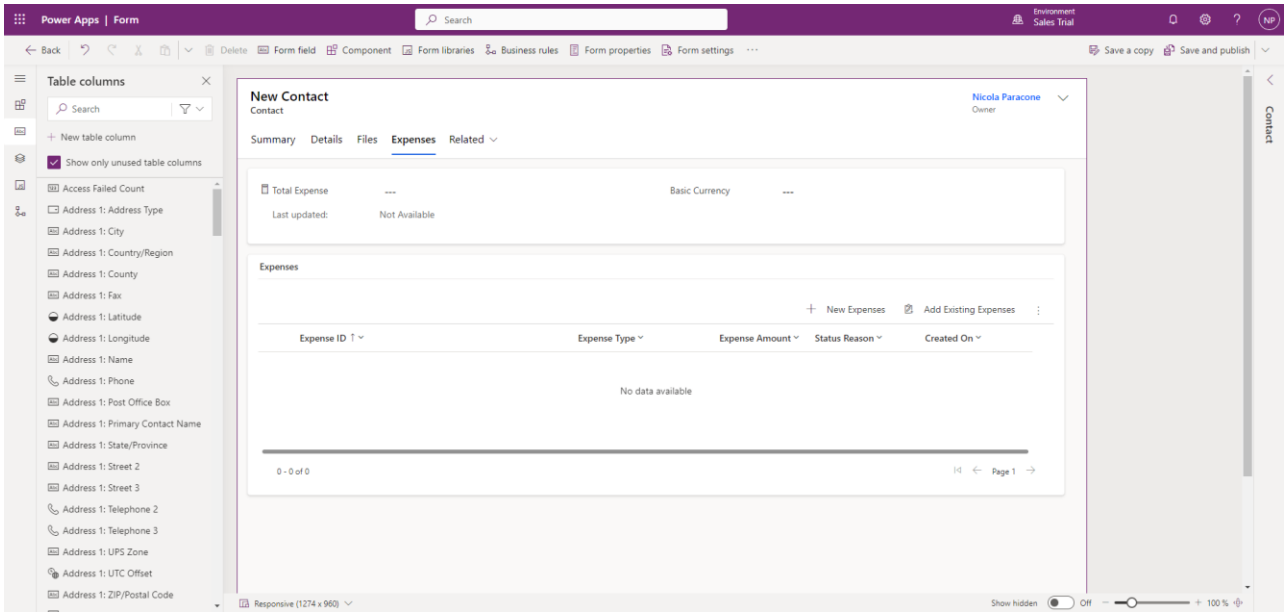
Hence, if we have an expense associated with Avery Howard, which has email address avery.howard@hotmail.com, and we compile the email field with the wrong email john.smith@gmail.com, we will obtain the alert "Email is not matching" message in Dynamics 365.



At this point, we create another column for the Contact entity.



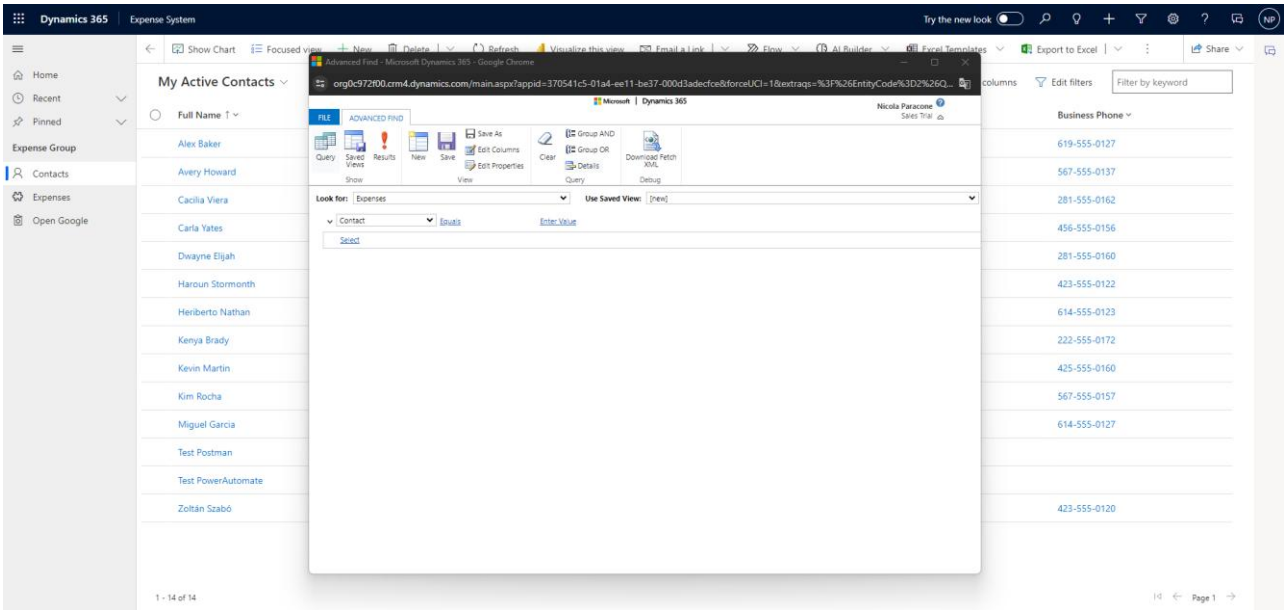
Both Total Expense and Basic Currency are added to a newly created 2-column section inside the Expenses tab.



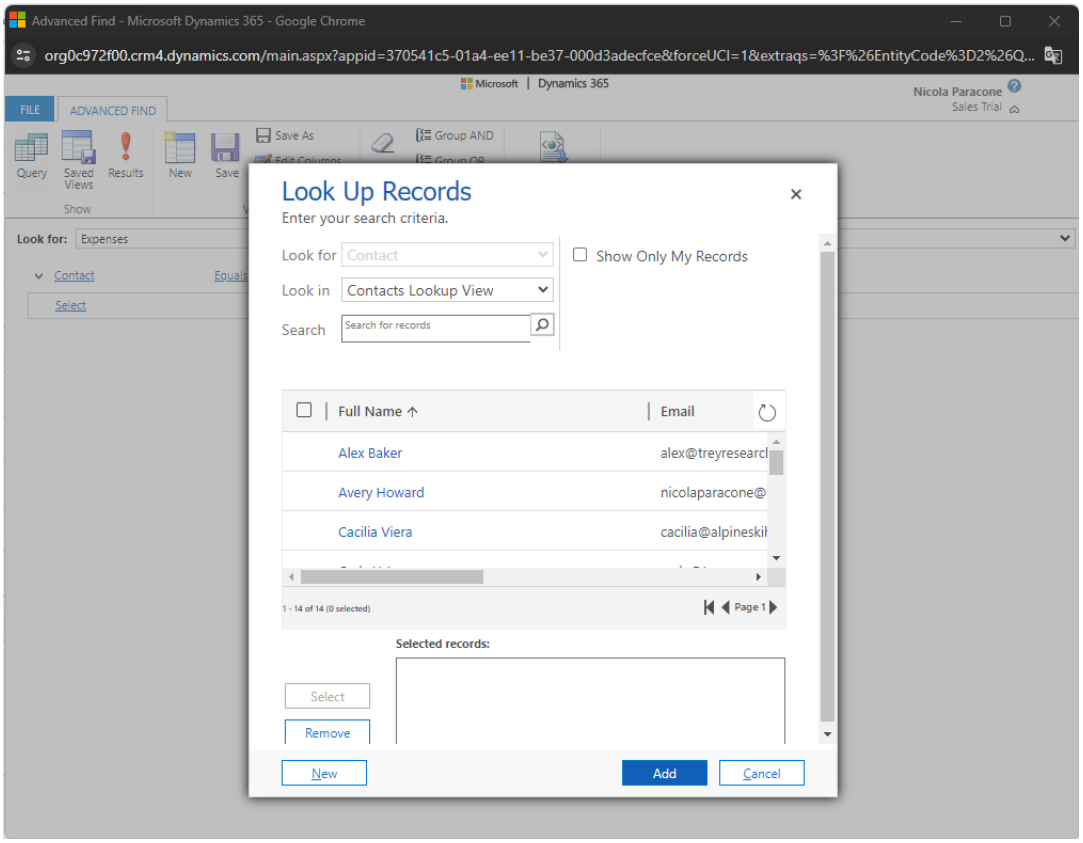
The intention is to retrieve all the expenses of a contact and store the total amount into the Basic Currency field. Summarizing, our requirement is: on change of Expense Amount, get the contact Id, retrieve all the expenses related to that particular contact, calculate the sum of all these expenses, and update it back to the contact.

Obviously, this operation will end with the result which we obtained with the rollup field Total Expense.

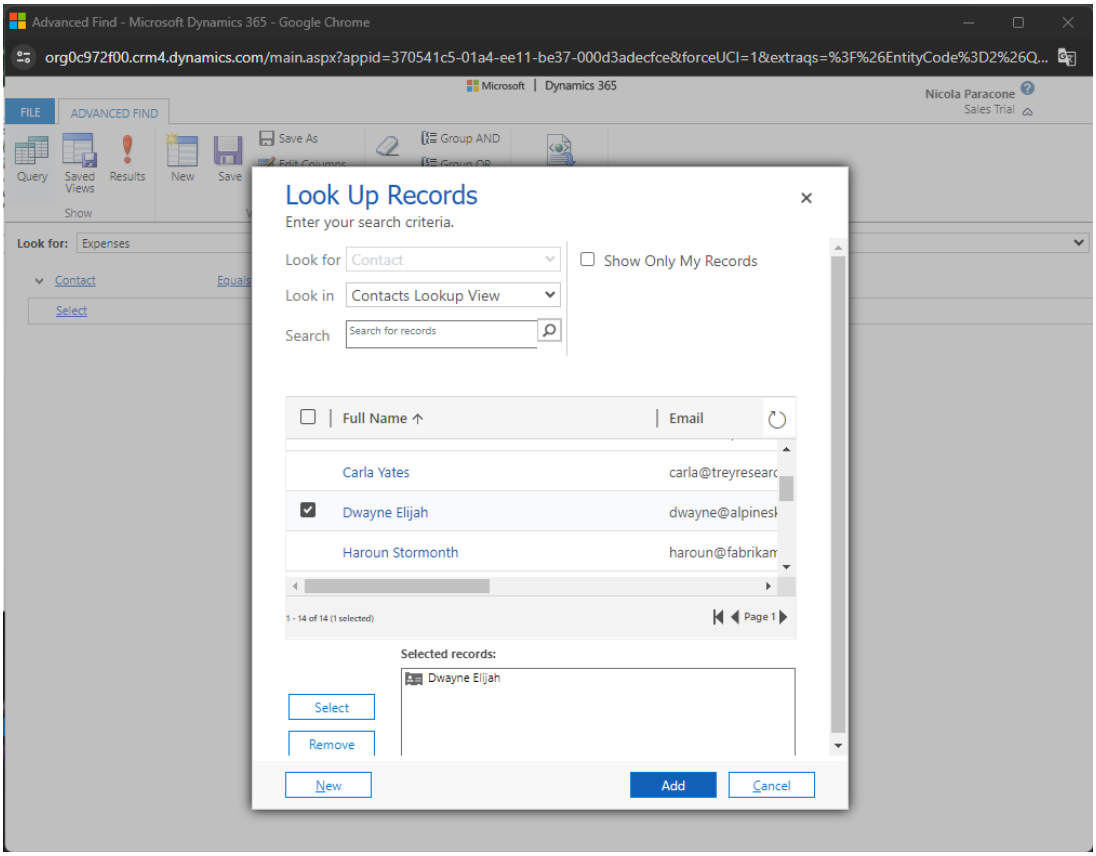
Inside Dynamics 365 click on the Advanced Find icon.



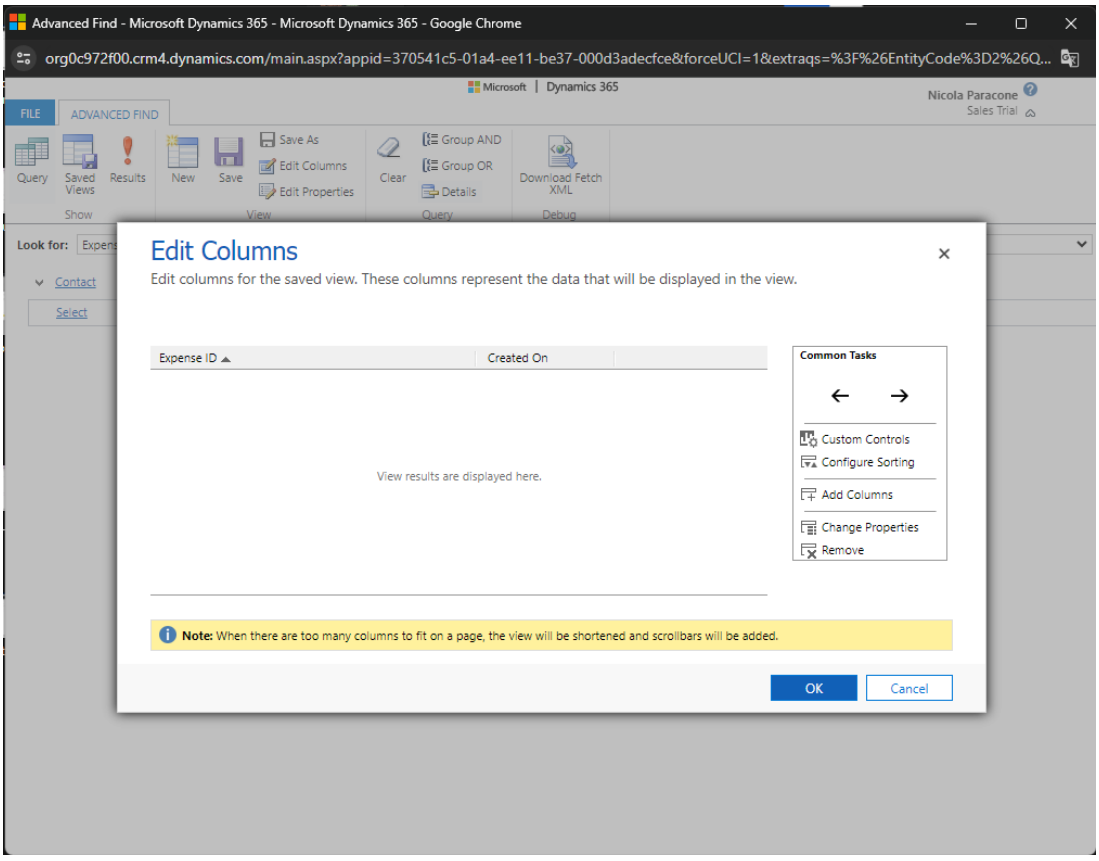
In the Look for menu select Expenses. Then choose Contact in the blank search box and click on Enter Value. The following window will pop up.



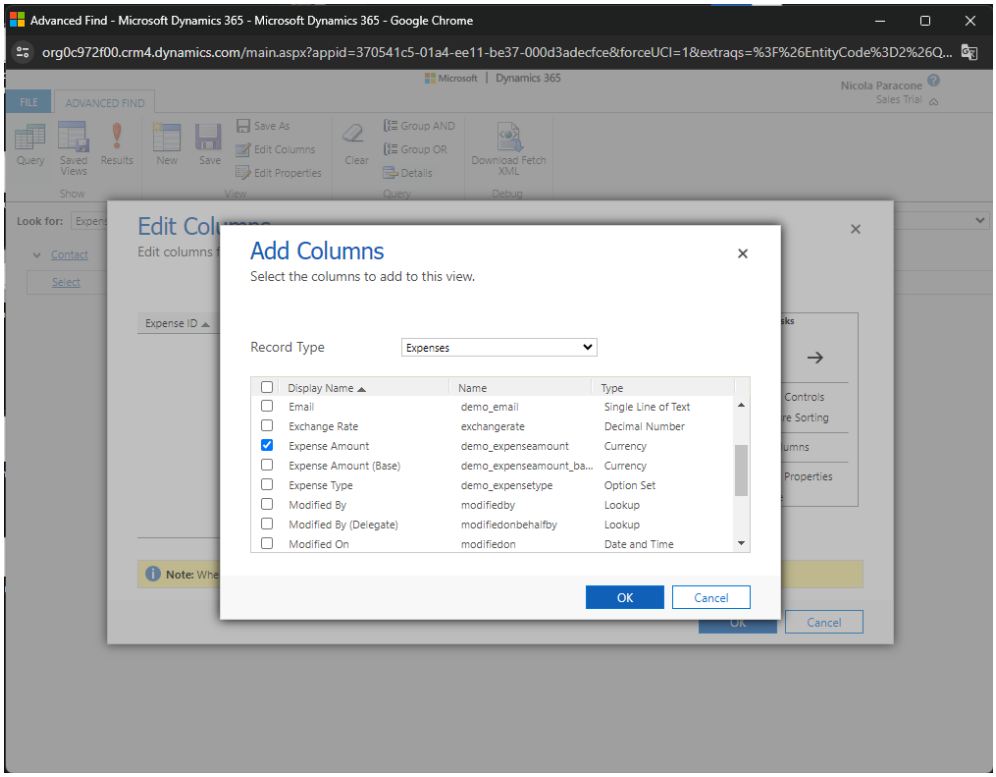
Here we select the contacts we want to retrieve.



Once back to the Advanced Find window, we need to specify which field we are interested in. For that we click on Edit Columns.

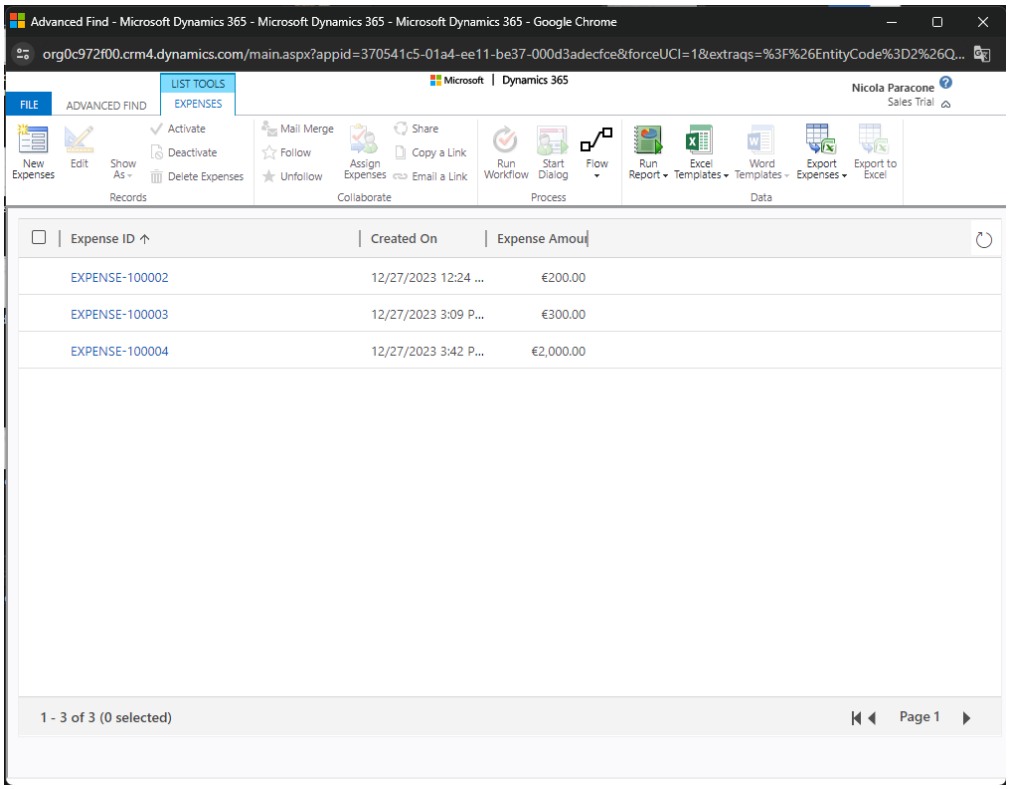


Click on Add Columns in the right panel and choose Expense Amount.



At this point the query is ready, and we can proceed by clicking on the red exclamation mark symbol called Results.

This is the general procedure; to continue, however, we select a contact which has already some expenses, for example Avery Howard.



Going back to the Advanced Find window, we click on the Download Fetch XML icon and save the XML file. This will contain something similar to what is shown in the following figure.

```
<fetch version="1.0" output-format="xml-platform" mapping="logical" distinct="false">
  <entity name="demo_expenses">
    <attribute name="demo_expensesid" />
    <attribute name="demo_name" />
    <attribute name="createdon" />
    <attribute name="demo_expenseamount" />
    <order attribute="demo_name" descending="false" />
    <filter type="and">
      <condition attribute="demo_contact" operator="eq" uiname="Avery Howard" uitype="contact" value="{79AE8582-84BB-EA11-A812-000D3A8B3EC6}" />
    </filter>
  </entity>
</fetch>
```

Navigate to https://learn.microsoft.com/en-us/power-apps/developer/data-platform/use-fetchxml-construct-query for a reference about querying data using FetchXML. In addition, navigate also to https://forwardforever.com/fetch-xml-the-hidden-gem-of-flow/ for a more friendly and operative description.

Basically, this file contains a request for information expressed in a formal manner and filtered according to our own criteria. In our example, however, the query is built only on single contact. What we need is instead a query dynamically adapting to the contact we open.

Editing the XML file as in the following figure is sufficient for our purpose.

```
<fetch version="1.0" output-format="xml-platform" mapping="logical" distinct="false">
  <entity name="demo_expenses">
    <attribute name="demo_expensesid" />
    <attribute name="demo_name" />
    <attribute name="createdon" />
    <attribute name="demo_expenseamount" />
    <order attribute="demo_name" descending="false" />
    <filter type="and">
      <condition attribute="demo_contact" operator="eq" uitype="contact" value="" />
    </filter>
  </entity>
</fetch>
```

Since we will use the content of this file as the value in double quotes of a variable inside our JavaScript resource, the first thing to do is to replace all the double quotes with single quotes.

Inside the editor click Ctrl+H for the find and replace functionality.

```
<fetch version='1.0' output-format='xml-platform' mapping='logical' distinct='false'>
  <entity name='demo_expenses'>
    <attribute name='demo_expensesid' />
    <attribute name='demo_name' />
    <attribute name='createdon' />
    <attribute name='demo_expenseamount' />
    <order attribute='demo_name' descending='false' />
    <filter type='and'>
      <condition attribute='demo_contact' operator='eq' uitype='contact' value='' />
    </filter>
  </entity>
</fetch>
```

This, however, is not sufficient since we have to append every line in order to have a correct string. Again, in the editor, press Ctrl+H and replace every < with "<. Then replace every > with ">+.

```
"<fetch version='1.0' output-format='xml-platform' mapping='logical' distinct='false'>"+
  "<entity name='demo_expenses'>"+
    "<attribute name='demo_expensesid' />"+
    "<attribute name='demo_name' />"+
    "<attribute name='createdon' />"+
    "<attribute name='demo_expenseamount' />"+
    "<order attribute='demo_name' descending='false' />"+
    "<filter type='and'>"+
      "<condition attribute='demo_contact' operator='eq' uitype='contact' value='' />"+
    "</filter>"+
  "</entity>"+
"</fetch>"
```

Notice that we already deleted the + at the end on the last line.

Navigate to https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/reference/xrm-webapi/retrievemultiplerecords to learn how to retrieve multiple records in a single action.

The following is the code which will calculate and store the result into the Basic Currency field.

```
async function updateTotalExpense(executionContext) {
    debugger;
    var formcontext = executionContext.getFormContext();
    var contactID = formcontext.getAttribute("demo_contact").getValue()[0].id; // This is for obtaining the contact id in the Expense entity
    var totalExpenseAmount = await getTotal(formcontext, contactID);
    var result = await updateContact(formcontext, contactID, totalExpenseAmount);
    console.log(result);
}

async function updateContact(formcontext, contactID, totalExpenseAmount) {
    var data = {
        "demo_basiccurrency": totalExpenseAmount
    };

    await Xrm.WebApi.updateRecord("contact", contactID, data).then(
        function success(result) {
            alert("Contact updated successfully with amount:" + totalExpenseAmount);
            console.log("Contact updated");
        },
        function (error) {
            alert(error.message);
            console.log(error.message);
        }
    );
}
```

```
async function getTotal(formcontext, contactID) {
    var totalExpense = 0;
    var fetchXML = "?fetchXml=<fetch version='1.0' output-format='xml-platform' mapping='logical' distinct='false'>" +
        "<entity name='demo_expenses'>" +
        "<attribute name='demo_expensesid' />" +
        "<attribute name='demo_name' />" +
        "<attribute name='createdon' />" +
        "<attribute name='demo_expenseamount' />" +
        "<order attribute='demo_name' descending='false' />" +
        "<filter type='and'>" +
        "<condition attribute='demo_contact' operator='eq' uitype='contact' value='" + contactID + "' />" +
        "</filter>" +
        "</entity>" +
        "</fetch>";

    await Xrm.WebApi.retrieveMultipleRecords("demo_expenses", fetchXML).then(
        function success(result) {
            for (var i = 0; i < result.entities.length; i++) {
                console.log(result.entities[i].demo_expenseamount);
                totalExpense = totalExpense + result.entities[i].demo_expenseamount;
            }
        },
        function (error) {
            console.log(error.message);
        }
    );
    return totalExpense;
}
```

An async function is like a special kind of function in JavaScript that helps us work with tasks that take some time to finish, such as loading data from the internet or doing calculations that may take a while.

Imagine we are baking cookies. While the cookies are in the oven, we don't just sit and wait. We might do other things, like cleaning up or preparing the next batch. An async function is a bit like that - it lets us program do other things while it's waiting for a particular task to complete.

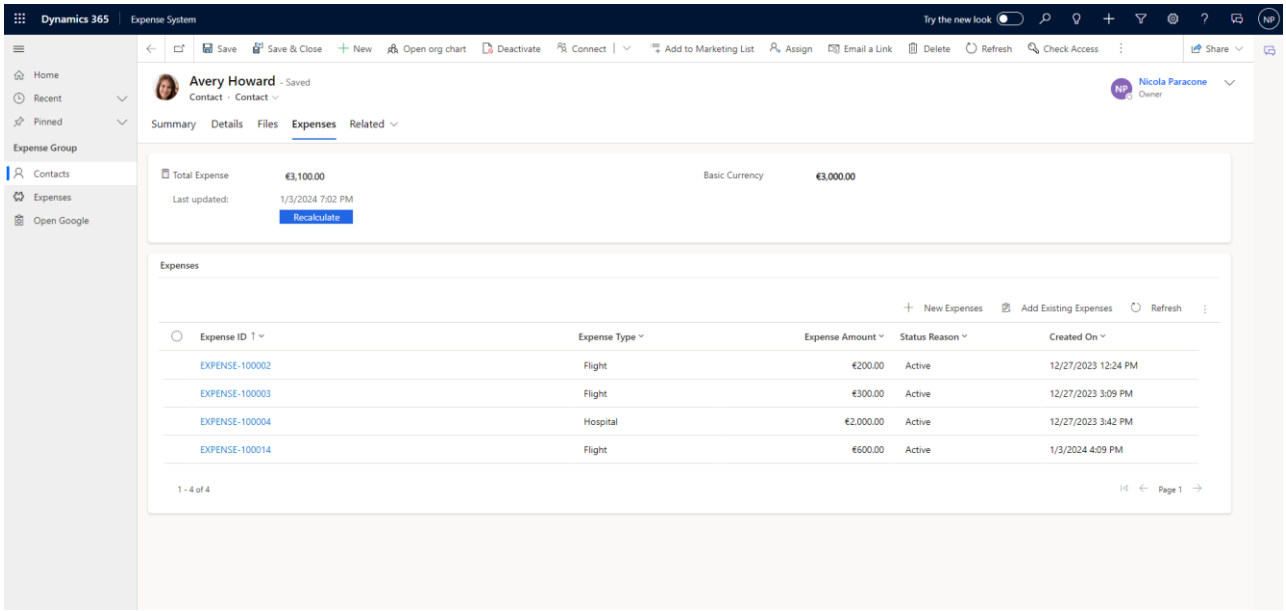
Here are the key points:

1. **Async Keyword**: When we see the word async before a function, it means that function can handle tasks that take time.

2. **Await Keyword**: Inside that function, when we see the word await before a task, it means the function will pause and let other things happen until that specific task is finished.

3. **Promise**: The async function always gives us a special object called a **Promise**. This Promise tells us when the task is done or if something went wrong.

Let us modify the Expense Amount of EXPENSE-100014 from 500 € to 600 €. The effect of the web resource on the contact record will be the following.



Basic Currency will store the total amount of expenses before the change of Expense Amount in one of the expenses.